CODITENT — Master Security & Project Defense Guide

Complete Technical Reference, Code Anatomy & Evaluator Correction Preparation

PROJECT / SUBJECT	CANDIDATE	SECURITY SCORE	ARCHITECTURE
CODITENT (ft_transcendence)	**ahabibi- (Abderrahmane Habibi)**	**Full Modules Completed (100%)**	**FastAPI + Next.js + Nginx + Vault + Redis**

🎯 **Evaluation ("Correction") Purpose:** This guide contains everything you need to know and explain during your peer defense. It details **all mandatory subject requirements**, **official optional security modules implemented**, **advanced features created beyond the subject**, **exact file paths and lines modified**, core cybersecurity concepts, and live terminal demonstration commands.

1. High-Level System Architecture & Defense Summary

CODITENT is a recruitment and assessment platform built as a secure microservices architecture orchestrated via Docker Compose. As the **Security Engineer** of the project, you implemented multi-layered defense-in-depth across the network, application, identity, and container layers.

🛡️ Multi-Layer Defense-in-Depth Architecture

LAYER	COMPONENT / TOOL	ROLE & RESPONSIBILITIES	STATUS
1. Perimeter & TLS	`Nginx 1.25` Reverse Proxy	Terminates TLS v1.2 / v1.3 with high-grade ciphers, forces HTTP → HTTPS 301 redirection, applies rate-limiting zones (10r/s API, 5r/s Auth).	ACTIVE
2. Web Firewall	`ModSecurity v3 + OWASP CRS`	Inspects incoming HTTP payloads, blocks SQL Injection (SQLi), Cross-Site Scripting (XSS), and Remote Code Execution (RCE) at the gateway.	ACTIVE
3. Secrets Storage	`HashiCorp Vault 1.15.0`	Key-Value v2 cryptographic engine storing sensitive production API keys and database tokens, preventing secrets in git.	ACTIVE
4. Identity & Auth	FastAPI + `passlib` + `PyOTP`	Bcrypt password hashing (work factor 12), JWT with high-precision UTC `iat`, TOTP RFC 6238 2FA with hashed backup codes.	ACTIVE
5. Session & Anomaly	`Redis 7` + `device_security.py`	Device fingerprinting (UA + IP subnet hashing), atomic Redis sets, Resend security email alerts, and global token revocation ("Logout all devices").	ACTIVE
6. Privacy & Rights	`app/routers/gdpr.py`	Article 20 Data Portability (JSON structured export) and Article 17 Right to Erasure (cascade account & personal data deletion).	ACTIVE
7. Container Isolation	`Dockerfile` (Non-Root User)	Process sandboxing under unprivileged system user `appuser:appgroup` (UID/GID 10001), preventing container breakout escalations.	ACTIVE

2. Subject Requirements Compliance (Chapter III & Chapter IV)

Below is the complete mapping of mandatory and optional subject requirements according to the 42 / ft_transcendence project specification:

2.1 Mandatory Security Requirements (Chapter III.3)

SUBJECT REQUIREMENT	SUBJECT RULE	HOW YOU IMPLEMENTED IT
HTTPS Enforcement	All client-to-backend traffic MUST be TLS-encrypted. Internal Docker traffic may be HTTP.	Configured Nginx reverse proxy on port 443 with TLS 1.2/1.3, SSL certificates auto-generated on container boot in `nginx/entrypoint.sh`, and 301 redirect on port 80.
Environment & Secrets	Zero credentials in Git repository; `.env` ignored; `.env.example` with placeholder values provided.	All secrets isolated in local `.env`; verified with Git audit; added HashiCorp Vault client (`apps/api/app/core/vault.py`) to retrieve runtime secrets.
Password Hashing	Passwords must be securely hashed and salted; no plaintext.	Used `passlib.context.CryptContext` with `bcrypt` algorithm. Passwords are automatically salted and computationally salted. Added `max_length=128` to prevent CPU ReDoS attacks.
Double-Sided Input Validation	All forms and fields must be validated on both Frontend and Backend.	**Backend:** Strict Pydantic v2 schemas in `apps/api/app/schemas.py` enforcing regex, types, and min/max lengths. **Frontend:** Zod validation schemas in `apps/web/src/app/(auth)/...` matching backend limits.
Container Security	Must launch via single command (`docker compose up --build`).	Clean orchestration in root `docker-compose.yml`; unprivileged `USER appuser` (UID 10001) in Dockerfile.

2.2 Official Modules Implemented (Chapter IV)

MODULE NAME	MODULE CATEGORY	POINTS	IMPLEMENTATION HIGHLIGHTS IN CODITENT
WAF / ModSecurity + HashiCorp Vault	Cybersecurity (IV.5)	**2 Pts (Major)**	Nginx compiled with ModSecurity v3 and OWASP Core Rule Set (CRS). HashiCorp Vault KV v2 running on port 8200 with dynamic REST client syncing secrets at startup.
Two-Factor Authentication (2FA)	User Management (IV.3)	**1 Pt (Minor)**	TOTP RFC 6238 implementation using `pyotp`. Generates QR codes (Base64 PNG), 8 SHA-256 hashed one-time backup recovery codes, 5-min temporary MFA token challenge.
OAuth 2.0 Integration	User Management (IV.3)	**1 Pt (Minor)**	Google and LinkedIn OAuth SSO with state token validation, profile synchronization, and automated account linking.
Advanced Permissions (RBAC)	User Management (IV.3)	**2 Pts (Major)**	Multi-tier role authorization in `dependencies.py`: `PLATFORM_ADMIN`, `COMPANY_USER` (with sub-roles: `OWNER`, `ADMIN`, `HR`, `RECRUITER`), and `CANDIDATE`.
GDPR Compliance Features	Data & Analytics (IV.8)	**1 Pt (Minor)**	Article 20 Data Portability (`POST /auth/gdpr/export`) returning complete user JSON archive; Article 17 Right to Erasure (`DELETE /auth/gdpr/delete`) with password verification.

3. Additional Advanced Implementations (Above & Beyond Subject)

To achieve enterprise-grade security and stand out during evaluation, you engineered four advanced proactive security mechanisms:

1. Device Fingerprinting & Email Alerts

Files: `apps/api/app/services/device_security.py`

- **Algorithm:** Generates a SHA-256 hash combining normalized User-Agent and client IP subnet (IPv4 `/24` or IPv6 `/64`).
- **Redis Deduplication:** Uses atomic `SADD user:{user_id}:known_devices <hash>` with a 90-day TTL.
- **Email Notification:** On detection of an unrecognized device, asynchronously fires a security alert email via Resend containing client browser, operating system, IP, and UTC login time.
- **Race-Condition Safe:** Concurrent logins from the same new device only send one alert due to atomic Redis sets.

2. Global Session Revocation ("Log Out All Devices")

Files: `apps/api/app/routers/auth.py`, `dependencies.py`, `jwt.py`

- **Stateless JWT Problem:** Standard JWTs cannot be revoked until their expiration time elapses.
- **The Solution:** Injected sub-second precision `iat` (issued at) claim into every JWT access token in `jwt.py`.
- **Revocation Trigger:** `POST /auth/logout-all-devices` sets Redis key `user:{id}:revoked_before = timestamp` with 30-day TTL.
- **Verification Guard:** `get_current_user` in `dependencies.py` checks if `token_iat ≤ revoked_before`. If true, returns `HTTP 401 Unauthorized` immediately.
- **Fail-Closed Design:** If Redis is unreachable, requests fail with `HTTP 503` instead of bypassing revocation checks.

3. Dual-Layer Rate Limiting & Anti-Brute-Force

Files: `nginx/nginx.conf`, `apps/api/app/limiter.py`

- **Gateway Level (Nginx):** `limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;` `limit_req_zone $binary_remote_addr zone=auth_limit:10m rate=5r/s;`
- **Application Level (FastAPI):** Integrated `slowapi` with Redis backend.
  - `/auth/login`: 5 requests / minute
  - `/auth/2fa/verify`: 10 requests / minute
  - `/auth/gdpr/export`: 2 requests / minute
  - `/auth/logout-all-devices`: 5 requests / minute
- **Defense:** Completely neutralizes credential stuffing and automated password dictionary attacks.

4. Non-Root Process Hardening & Trivy Scans

Files: `apps/api/Dockerfile`, `scripts/trivy_scan.sh`

- **Container Isolation:** Created dedicated system user and group:

```
RUN addgroup --system --gid 10001 appgroup   && adduser --system --uid 10001      --ingroup appgroup appuser   && chow
USER appuser
```

- **Breakout Defense:** Even if an attacker finds an RCE exploit in Python, they cannot access host devices or root privileges.
- **Automated CVE Scanning:** Created `scripts/trivy_scan.sh` to scan image layers for known vulnerabilities.

4. Complete File, Directory & Code Anatomy

Here is every file in the repository that you created or modified, including the exact directory, functionality, and lines of code:

DIRECTORY & FILE PATH	ACTION	KEY CODE PATTERNS & LOGIC ADDED
`nginx/nginx.conf`	CREATED	Configured TLS v1.2/1.3, ciphers, 301 HTTPS redirect, ModSecurity WAF directive, rate-limiting zones (`api_limit` , `auth_limit`), upstream proxies (`web:3000` , `api:8001`), and 7 OWASP security headers.
`nginx/entrypoint.sh`	CREATED	Shell script checking `/etc/nginx/ssl/cert.pem` ; automatically invokes `openssl req -x509 -nodes -days 365 -newkey rsa:2048` if certificates are missing on startup.
`nginx/Dockerfile`	CREATED	Multi-stage Nginx image building ModSecurity v3 connector and copying configuration and entrypoint scripts.
`docker-compose.yml`	MODIFIED	Configured 6 coordinated services: `proxy` (Nginx HTTPS :8443), `api` (FastAPI :8001), `web` (Next.js :3001), `redis` (:6380), `vault` (:8200), and `worker` (Celery).
`apps/api/Dockerfile`	MODIFIED	Hardened Python 3.12-slim base; created unprivileged system user `appuser:appgroup` (UID 10001); switched context using `USER appuser` .
`apps/api/app/core/vault.py`	CREATED	Implemented `VaultClient` class: `health_check()` querying `/v1/sys/health` , `write_secrets()` , `read_secrets()` from `/v1/secret/data/coditent` , and `get_secret()` with environment variable fallback.
`apps/api/app/services/device_security.py`	CREATED	`get_device_fingerprint()` hashing User-Agent and `/24` IP subnet; `parse_user_agent_friendly()` ; `check_and_notify_new_device()` with atomic Redis `SADD` and async HTML Resend email dispatch.
`apps/api/app/services/two_factor.py`	CREATED	TOTP engine: `generate_totp_secret()` , `get_totp_uri()` , `generate_qr_code_base64()` , `verify_totp_code()` via `pyotp.TOTP` , and `generate_backup_codes()` hashing 8 random codes with SHA-256.
`apps/api/app/routers/two_factor.py`	CREATED	Endpoints: `GET /status` , `POST /setup` (returns QR code & secret), `POST /enable` (validates code & returns backup codes), `POST /disable` (requires password + code), `POST /verify` (validates 5-min MFA token and issues full JWT + trusted device cookie).
`apps/api/app/routers/gdpr.py`	CREATED	Endpoints: `POST /export` (Article 20 Data Portability, exports user info, candidate profile, and job applications as JSON), `DELETE /delete` (Article 17 Right to Erasure, validates password and permanently cascade-deletes user records).
`apps/api/app/routers/auth.py`	MODIFIED	Added `POST /logout-all-devices` updating Redis revocation timestamp; hooked `check_and_notify_new_device` into regular login and 2FA challenge.
`apps/api/app/dependencies.py`	MODIFIED	Updated `get_current_user` : checks Redis key `user:{id}:revoked_before` against token `iat` timestamp; rejects revoked sessions; enforces role guards (`require_candidate` , `require_company_user` , `require_platform_admin`).
`apps/api/app/utils/jwt.py`	MODIFIED	Updated `create_access_token` to automatically inject microsecond-precision UTC `iat` timestamp into token payload for revocation comparison.
`apps/api/app/schemas.py`	MODIFIED	Added bounded validation schemas: `TwoFactorSetupOut` , `TwoFactorVerifyRequest` , `GDPRExportOut` , `GDPRDeleteRequest` , and `max_length=128` on password requests to stop ReDoS.
`scripts/trivy_scan.sh`	CREATED	Automated bash script for vulnerability scanning container images using Trivy CLI.

5. Core Concepts, Definitions & Technical Knowledge (Must-Know for Correction)

During your correction, the evaluator will ask you technical questions about the security terms and concepts used in the code. Review these definitions and explanations:

1. What is the difference between a Reverse Proxy (Nginx) and a Web Application Firewall (ModSecurity)?

An **Nginx Reverse Proxy** operates at the transport and network layer: it handles SSL/TLS termination, request routing, load balancing, compression, and basic IP rate limiting. However, it does not inspect the contents of HTTP request bodies.
A **Web Application Firewall (ModSecurity WAF)** operates at the application layer (Layer 7). It analyzes HTTP requests, headers, and body payloads against known attack signatures (using the OWASP Core Rule Set) to detect and block SQL Injections, Cross-Site Scripting (XSS), Local File Inclusions (LFI), and malicious bots before they ever reach FastAPI.

2. How does Two-Factor Authentication (TOTP - RFC 6238) work under the hood?

TOTP stands for **Time-Based One-Time Password**. It is an extension of HMAC-Based One-Time Password (HOTP - RFC 4226):

1. During setup, the server generates a cryptographically random Base32 shared secret key (e.g., 32 characters) and shares it with the user via a QR code (`otpauth://totp/...`).
2. Both the server and the authenticator app compute a counter based on the current Unix timestamp: \(T = \lfloor rac{ ext{CurrentTime} - T_0}{X} floor \), where \(X = 30 \) seconds.
3. Both sides compute an HMAC-SHA1 hash using the shared secret and counter \(T \).
4. Dynamic truncation extracts a 6-digit numerical code. Because both sides share the secret and the same 30-second time window, they arrive at identical 6-digit codes without sending any secret over the network.
5. To handle minor clock drift between devices, our verification allows a ±1 time-step window (validating current, previous, or next 30-second slice).

3. How does Stateless JWT Revocation ("Log Out All Devices") work using Redis?

Standard JSON Web Tokens are **stateless**: once signed by the server's private secret, they remain cryptographically valid until the `exp` (expiration) timestamp passes, meaning deleting a token on the frontend does NOT invalidate it if stolen.
To implement global revocation without storing every single active token in a database:

1. Every JWT issued includes an `iat` (issued-at) timestamp representing the exact second it was created.
2. When a user clicks "Log Out All Devices", the backend writes a timestamp into Redis: `user:{user_id}:revoked_before = current_timestamp` with an expiry equal to the token lifetime (e.g., 30 days).
3. Whenever any request comes in, our FastAPI dependency (`get_current_user`) extracts the token, verifies the cryptographic signature, and does an ultra-fast \(O(1)\) lookup in Redis.
4. If `token.iat ≤ revoked_before`, the server immediately rejects the request with `401 Unauthorized`. New logins issued after that timestamp will have `token.iat > revoked_before` and succeed normally.

4. Why do we store passwords with Bcrypt and enforce a 128-character limit?

Bcrypt is an adaptive, salted key-derivation function based on the Blowfish cipher. It incorporates:

- **Automatic Salting (128 bits):** Generates a unique cryptographic salt for every user, defeating precomputed Rainbow Table attacks.
- **Configurable Work Factor:** We use work factor 12 (\(2^{12} = 4096\) iterations), making brute-force dictionary cracking computationally prohibitive on GPUs.

The 128-character limit: Bcrypt has a known algorithmic quirk where inputs longer than 72 bytes are truncated, and extremely long strings (e.g., 50,000 characters) cause high CPU load when hashing. Enforcing `max_length=128` protects the server from **ReDoS / CPU-Exhaustion Denial of Service** attacks where an attacker sends massive passwords to freeze backend workers.

5. What is HashiCorp Vault and why is it superior to hardcoded environment variables?

In simple projects, secrets are stored in a `.env` file. However, in production systems:

- Files on disk can be exposed through directory traversal vulnerabilities, backups, or Docker image inspections.
- Plaintext env vars can leak into crash reports, logs, and child processes.

HashiCorp Vault is a centralized secrets manager that encrypts secrets at rest using AES-256-GCM. It supports dynamic secrets, secret leasing, audit logging (recording exactly which service read a secret and when), and programmatic secret rotation via API (`/v1/secret/data/coditent`).

6. What HTTP Security Headers did you configure and what attack does each header prevent?

HEADER	CONFIGURATION VALUE	ATTACK / RISK MITIGATED
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains; preload`	**SSL Stripping & Man-in-the-Middle (MitM):** Forces browsers to communicate strictly over HTTPS for 1 full year.
`X-Frame-Options`	`DENY`	**Clickjacking:** Prohibits embedding the web application inside `<iframe>` tags on external websites.
`X-Content-Type-Options`	`nosniff`	**MIME-Type Sniffing:** Prevents browsers from interpreting executable scripts disguised as images or text files.
`Content-Security-Policy`	`default-src 'self' ...`	**Cross-Site Scripting (XSS) & Data Injection:** Restricts which domains and sources can load scripts, stylesheets, and fonts.
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	**Browser Feature Abuse:** Blocks unauthorized browser APIs (webcam, mic, GPS) from being accessed by malicious third-party scripts.
`Referrer-Policy`	`strict-origin-when-cross-origin`	**Data Leakage in URLs:** Truncates sensitive query parameters from the HTTP Referer header when navigating to external origins.

7. Why is running Docker containers as a Non-Root user critical?

By default, processes in Docker run as `root` (UID 0). While Linux namespaces isolate the container, the container root shares the exact same Linux kernel as the host machine. If an attacker discovers a Remote Code Execution (RCE) vulnerability or a Linux kernel privilege escalation exploit (e.g., Dirty COW, runc CVEs), running as root allows them to escape the container and compromise the entire host operating system.
By enforcing `USER appuser (UID 10001)` in our Dockerfile, the process operates with zero unprivileged permissions on both the container filesystem and host kernel.

6. Live Evaluation ("Correction") Step-by-Step Defense Script

When the evaluator sits next to you and asks you to demonstrate your security implementation, follow this exact sequence:

> 💡 **Tip for Defense:** Keep one terminal open for Docker logs and curl commands, and another terminal or browser tab open for the web interface. Walk them through layer by layer!

Step 1: Demonstrate SSL/HTTPS, ModSecurity & Security Headers

Show the evaluator that the Nginx reverse proxy enforces HTTPS and returns all OWASP security headers:

```
curl -k -I https://localhost:8443/api/health
```

Point out to evaluator:

- Status is `HTTP/2 200`.
- Presence of `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`.
- Open `http://localhost:8080/` → Show them that it returns `HTTP/1.1 301 Moved Permanently` redirecting to HTTPS!

Step 2: Demonstrate Non-Root Container Execution

Prove that the API process is completely sandboxed without root privileges:

```
docker exec coditent-api whoami
# Output: appuser

docker exec coditent-api id
# Output: uid=10001(appuser) gid=10001(appgroup) groups=10001(appgroup)
```

Step 3: Demonstrate HashiCorp Vault Secrets Storage

Show that the Vault service is running and managing credentials:

```
curl -s http://localhost:8200/v1/sys/health | jq .
# Output: {"initialized":true,"sealed":false,"standby":false,"performance_standby":false,"replication_performance_mode":"disabled",...}
```

Explain: *"We configured a Vault client in `apps/api/app/core/vault.py`. When FastAPI boots, it authenticates with Vault and reads/writes runtime keys to the KV v2 engine at `secret/data/coditent`."*

Step 4: Demonstrate Two-Factor Authentication (2FA / TOTP)

Explain the 4-phase lifecycle:

1. **Setup (`POST /auth/2fa/setup`):** Generates TOTP secret and returns base64 QR code. User scans it with Google Authenticator.
2. **Enable (`POST /auth/2fa/enable`):** User submits 6-digit code. Server enables 2FA and returns 8 emergency recovery backup codes (hashed with SHA-256 before saving to DB).
3. **Login Challenge:** When an enabled user enters their password at `/auth/login`, the backend does NOT issue an access token. Instead, it returns `{"require_2fa": true, "mfa_token": "..."}` (a short-lived 5-minute token).
4. **Verification (`POST /auth/2fa/verify`):** User submits code + MFA token. Server verifies TOTP or consumes one backup code, sets trusted device cookie, and issues full access token.

Step 5: Demonstrate GDPR Data Portability & Right to Erasure

Demonstrate both mandatory European privacy rights:

```
# 1. Data Portability (Article 20): Export complete structured JSON of user profile and applications
curl -X POST https://localhost:8443/api/auth/gdpr/export   -H "Authorization: Bearer <TOKEN>" -k | jq .

# 2. Right to Erasure (Article 17): Secure account deletion with password verification
curl -X DELETE https://localhost:8443/api/auth/gdpr/delete   -H "Authorization: Bearer <TOKEN>"   -H "Content-Type: application/json"   -d '{"password": "Val
```

Explain: *"Notice that deleting an account requires password re-authentication. The deletion cascade-purges candidate profiles, job applications, and credentials completely from PostgreSQL."*

Step 6: Demonstrate New Device Alert & Global Logout

Highlight your custom advanced features:

- **Device Alert:** *"When I log in from a new browser or IP, `device_security.py` hashes the user-agent and IP subnet, checks Redis, and triggers an email alert via Resend informing the user of unrecognized activity."*
- **Logout All Devices:** *"Calling `POST /auth/logout-all-devices` writes the current timestamp to Redis. Every subsequent request checks the token's `iat` claim against this timestamp. Any token issued earlier is instantly rejected with 401 Unauthorized."*

7. Evaluator Quick-Reference Q&A Cheat Sheet

POTENTIAL EVALUATOR QUESTION	YOUR DIRECT, CONFIDENT ANSWER
"Why did you use Bcrypt instead of SHA-256 or MD5 for passwords?"	"MD5 and SHA-256 are general cryptographic hash functions designed to be extremely fast (billions of hashes per second on GPUs), which makes them vulnerable to brute-force cracking. Bcrypt is a dedicated key-derivation function that includes built-in 128-bit salting and a configurable work factor (we use 12), intentionally making it slow and computationally expensive for attackers."
"Where are the SSL certificates stored and how are they generated?"	"They are located in `nginx/ssl/cert.pem` and `key.pem`. To satisfy the single-command startup requirement without requiring manual setup on evaluator machines, our `nginx/entrypoint.sh` script checks for certificate presence and automatically generates 2048-bit RSA self-signed certificates on container boot if absent."
"Can an attacker bypass 2FA by directly hitting private API endpoints?"	"No. The login endpoint only returns a short-lived `mfa_pending` token with `type: mfa_pending`. The dependency `get_current_user` explicitly checks `payload.get('type') is not None` and rejects it if used as a regular access token. Only the `/auth/2fa/verify` endpoint can issue a full access token."
"What happens if Redis goes down while checking revoked tokens?"	"Our code implements a **Fail-Closed** security design. If the Redis client raises a connection exception in `dependencies.py`, we return `503 Service Unavailable` rather than letting the request pass. In security, failing closed protects against unauthorized access during infrastructure outages."
"How does Double-Sided Input Validation prevent attacks?"	"Client-side validation with Zod improves user experience by giving instant feedback, but can easily be bypassed with `curl` or Postman. Therefore, our backend enforces strict Pydantic schemas that sanitize strings, enforce character bounds, and reject malformed payloads before they reach business logic or database queries."